We start by calling `search(A, (-1.5, -2))`, where node `A` is the root of the k-d tree, as shown in the drawing. Since `A` is not null, at line #2 the condition fails, and we can compare `A.point`, which is the tuple `(0, 5)`, to our target, at line #3: they obviously don't match, so we move onto line #4 and use the `compare` helper function to check which direction we should take. `A.level` will evaluate to `0`, so we compare the first value in each of the tuples: `-1.5 < 0`, thus we traverse the left subtree and call `search(C, (-1.5, -2))`.

For this call, we repeat more or less the same steps, except this time `C.level` is equal to 1, and such we compare the second value in each tuple: `-2 < 6` so we still go left, calling `search(D, (-1.5, -2))`.

Once again, we go through lines #2, #3, and #4, and we take a left turn; only this time `D.left == null`, so we call `search(null, (-1.5, -2))`, which will return `null` at line #2. The execution back-traces through the call stack, and our original call will also return `null`, stating that the target point was not found on the k-d tree.

Figure 9.10 shows another example, calling `search(A, (2, -5))`: on the first call, conditions at lines #2 and #3 are false, as well as the condition at line #4, since `2 > 0`. This time, therefore, we take a right turn at node `A`, and recursively call `search(B, (2, -5))`, then in turn `search(E, (2, -5))`, for which the condition at line #3 is true (`E.point` matches target), and thus we finally return node `E` as the result of the original call.